LibanPost eShop — Security Guide & Hardening Checklist

Contents

1. Authentication . . . 2
   - Hardening checklist . . . 2
2. Authorization . . . 2
   - Hardening checklist . . . 2
3. Rate limiting . . . 2
   - Hardening checklist . . . 3
4. Security headers . . . 3
   - Hardening checklist . . . 3
5. Antiforgery . . . 3
   - Hardening checklist . . . 3
6. Payment webhook security . . . 3
   - Hardening checklist . . . 3
7. Image uploads . . . 4
   - Hardening checklist . . . 4
8. Data protection . . . 4
   - Hardening checklist . . . 4
9. Logging & audit . . . 4
   - Hardening checklist . . . 4
10. Configuration & secrets . . . 5
    - Hardening checklist . . . 5
11. Network & deployment . . . 5
    - Hardening checklist . . . 5
12. Quick verification commands . . . 5

Operator-facing reference for securing a deployment. Written against the application as it ships; verify against your own environment before relying on any single item.

1. Authentication

- **Mechanism**: cookie authentication (`Microsoft.AspNetCore.Authentication.Cookies`), 14-day sliding expiry, `HttpOnly`, `SameSite=Lax`.
- **Password storage**: PBKDF2-HMAC-SHA256, 210,000 iterations, constant-time comparison.
- **Lockout**: 5 failed attempts within 15 minutes locks the account.
- **Security stamp**: rotated on credential change; reset tokens are signed and expire.
- **Sign-in messages are generic** — no account enumeration via login errors.
- **Password reset**: signed, expiring token (Data Protection, `Email:PasswordResetMinutes`); the request step always reports success.

Hardening checklist

- [ ] Set `ASPNETCORE_ENVIRONMENT=Production`.
- [ ] Use HTTPS only; enable HSTS (the app emits it on HTTPS requests).
- [ ] Confirm cookie `Secure` policy is enforced by your reverse proxy / TLS termination.
- [ ] Rotate `Data Protection` keys via a persisted key ring (default is in-memory/ephemeral in containers — configure a shared key store for multi-instance).
- [ ] Enforce a strong password policy at registration (length, complexity) — review `AccountService` validation.
- [ ] Disable or remove demo accounts (`admin@libanpost.example`, `customer@libanpost.example`) before production.

2. Authorization

- Policies: `AdminOnly` (role `Admin`), `VendorOnly` / `CustomerOnly` (authenticated user).
- Shop membership is enforced in `VendorControllerBase`, not by policy — verify every vendor controller inherits it.
- Admin area routes are under `/Admin` with the `AdminOnly` policy applied globally via `AreaAuthorization` filters.

Hardening checklist

- [ ] Audit every controller action for `[Authorize]` / policy attributes.
- [ ] Verify a vendor cannot reach another shop's resources (returns 404, not 403, by design).
- [ ] Confirm soft-deleted products and withdrawn shops never leak through admin filters.

3. Rate limiting

Four named policies, fixed-window, partitioned by signed-in user or IP:

Policy	Default	Surface
`auth`	30 / 60s	sign-in, register, password reset
`checkout`	30 / 60s	placing an order
`review`	30 / 60s	posting reviews / voting
`webhook`	120 / 60s	provider callbacks (retry-tolerant)

Rejected requests return **429** with `Retry-After`, are logged, and do not disturb other policies. Disable entirely with `RateLimiting:Enabled = false`.

Hardening checklist

- [ ] Tune limits to your traffic profile; lower `auth` if brute-force is a concern.
- [ ] Put the app behind a reverse proxy that forwards the real client IP (`X-Forwarded-For`) so partitioning is by client, not by proxy.
- [ ] Monitor 429 rates; alert on sustained rejection.

4. Security headers

Emitted by middleware on every response:

- `X-Content-Type-Options: nosniff`
- `X-Frame-Options: SAMEORIGIN`
- `Referrer-Policy: strict-origin-when-cross-origin`
- `Permissions-Policy: geolocation=(), microphone=(), camera=()`
- `Strict-Transport-Security` (HTTPS only)
- `Content-Security-Policy-Report-Only` (permissive; tighten after auditing violations)

Hardening checklist

- [ ] Move CSP from report-only to enforced once violations are cleared.
- [ ] Add `X-XSS-Protection` only if supporting legacy browsers (modern browsers ignore it).
- [ ] Confirm headers are present on static files and error pages, not just MVC responses.

5. Antiforgery

- Global antiforgery: every POST without a valid token returns 400.
- AJAX endpoints (`/Cart/Summary`, `/Catalog/Suggest`) are intentionally exempt — document why.
- Checkout is idempotent via a per-form `IdempotencyKey` (uniquely indexed).

Hardening checklist

- [ ] Verify every form renders `@Html.AntiForgeryToken()` or uses `asp-action` (which adds it).
- [ ] Confirm antiforgery cookie is `HttpOnly` and `SameSite`.

6. Payment webhook security

- Signature verified **before** the body is trusted: raw request body read, `X-Payment-Signature` checked with HMAC-SHA256 over the raw bytes.
- Unverified deliveries recorded with `SignatureValid = 0` and answered **401**.
- Idempotent: every event written to `PaymentEvents`, keyed by `(Provider, EventId)`; redelivery returns `duplicate` and changes nothing.
- `MarkPaidAsync` is a no-op on already-paid orders and cannot revive cancelled/refunded ones.
- Unknown provider → 404; unknown order → recorded and answered 200 (no infinite retry).

Hardening checklist

- [ ] Set a strong, unique `Payments:WebhookSecret` (≥16 chars) in production; the app fails fast if it is blank/short.

- [ ] Restrict the webhook endpoint at the firewall to the provider's IP ranges where possible.
- [ ] Rotate the secret on suspected compromise; re-issue provider credentials.
- [ ] Monitor `PaymentEvents` for `SignatureValid = 0` rows (attack indicator).

7. Image uploads

- Allowed extensions: `.jpg .jpeg .png .gif .webp`. SVG deliberately rejected (script carrier).
- Magic bytes verified against extension (renamed scripts rejected).
- Per-file limit 5 MB; gallery cap 10 images per product.
- Files stored under a GUID name in a store-owned folder; no path traversal possible.
- Read is bounded; oversized uploads dropped without full buffering.
- A failed save removes any files written during that request (no orphans).
- Removing an image URL deletes the stored file unless another product references it.

Hardening checklist

- [ ] Keep `Images:Folder` outside the web root in production; serve via the configured `Images:RequestPath` (static files middleware).
- [ ] Add malware scanning on upload if accepting untrusted content.
- [ ] Back up the upload folder alongside the database.
- [ ] Set filesystem permissions so the app user can write only to the upload folder.

8. Data protection

- Orders snapshot product name, variant, SKU and unit price at sale time — history immutable.
- Products are soft-deleted; sold variants deactivated, not removed.
- Stock committed under optimistic concurrency (`RowVersion`); concurrent oversell raises `DbUpdateConcurrencyException` and retries.
- Checkout idempotent via per-form key + unique `Order.IdempotencyKey`.
- Every stock change recorded in the append-only `StockMovement` ledger.

Hardening checklist

- [ ] Enable database backups with point-in-time recovery.
- [ ] Restrict DB credentials to least privilege (no DDL in production runtime).
- [ ] Encrypt at rest (SQL Server TDE / SQLite file encryption where supported).

9. Logging & audit

- All logs go to console **and** the `Log` table; `Error`/`Critical` include full stack traces.
- Logging calls never throw or block the request (bounded queue, background writer).
- `AuditLog` records staff actions: shop decisions, order status changes, return decisions, refunds (including failed attempts), payouts.
- Audit references are loose (name + id, not FKs) — history survives row/account deletion.
- Audit writes are best-effort (logged, never undo the action).

Hardening checklist

- [ ] Set `Logging:Database:RetentionDays` > 0 to bound table growth.
- [ ] Restrict read access to the `Log` and `AuditLog` tables.
- [ ] Forward logs to a SIEM for long-term retention and alerting.

- [ ] Review `/Admin/Audit` regularly; export filtered views.

10. Configuration & secrets

- `appsettings.Local.json` (gitignored) holds real secrets; committed files carry placeholders.
- `Program.cs` loads the Local file last so it overrides.
- `Database:Provider` switches SQLite/SqlServer by config.
- `Payments:Provider` must be `sandbox` or `simulator` (or a registered gateway); unknown values throw at startup.

Hardening checklist

- [ ] Never commit `appsettings.Local.json` or real secrets.
- [ ] Use environment variables or a secret manager in production (`Database__Provider`, `Payments__WebhookSecret`, etc.).
- [ ] Fail fast on missing required secrets (already done for webhook secret in Production).
- [ ] Review `appsettings.json` for any leftover demo values before deploying.

11. Network & deployment

- Run behind a reverse proxy (IIS / nginx / Caddy) terminating TLS.
- Forward real client IP for rate-limit partitioning.
- Restrict `/health` if you do not want it public (or leave it for orchestrators).
- The bundled `simulator` payment gateway moves no money — register a real `IHostedPaymentGateway` for production.

Hardening checklist

- [ ] Enable HTTPS redirection (`UseHttpsRedirection` — already wired).
- [ ] Set `ASPNETCORE_FORWARDEDHEADERS_ENABLED` or configure forwarded headers middleware.
- [ ] Disable directory browsing and default error pages in production (custom error pages wired).
- [ ] Keep the OS and .NET runtime patched.

12. Quick verification commands

```
# Build
dotnet build Kwikby_MP.csproj

# Tests
dotnet test Tests/WebDS_Test.Tests.csproj -p:RemoveTestHostAppHost=true

# Health
curl http://localhost:5268/health
curl http://localhost:5268/health/live
curl http://localhost:5268/health/ready

# Headers
curl -I https://your-host/
```

This guide describes the controls as implemented. Where code and guide disagree, the code is authoritative.